

Plan de migración a ROS para navegación

VLMaps + YOLOE + Habitat-Sim → ROS para navegación y futura integración con HSR

Mario García Blasco

23 de abril de 2026 (revisión)

Chuleta operativa ROS/Gazebo

Esta sección se mantiene como lista viva de comandos prácticos para lanzar el *stack* ROS sin depender de menús. La idea es poder ejecutar siempre lo esencial a mano, primero desde el host para arrancar el contenedor y después desde dentro de `tfg-ros` para compilar, lanzar Gazebo o inspeccionar topics.

1. Arrancar el contenedor ROS desde el host.

```
cd /home/mario/tfg/vlmap-semantic-object-search-tfg/docker
docker compose up -d tfg-ros
docker exec -it tfg-ros bash
```

2. Compilar el workspace ROS dentro de tfg-ros.

```
cd /ros_ws
catkin build
```

3. Lanzar HSR oficial sólo en Gazebo.

```
roslaunch vlmap_bringup hsr_gazebo_only.launch gui:=true use_rviz:=false
```

4. Lanzar HSR oficial en Gazebo con RViz.

```
roslaunch vlmap_bringup hsr_gazebo_only.launch gui:=true use_rviz:=true
```

5. Lanzar HSR oficial en mundo simple y estático.

```
roslaunch vlmap_bringup hsr_gazebo_simple.launch
```

6. Lanzar HSR oficial en interior estático validado.

```
roslaunch vlmap_bringup hsr_gazebo_indoor.launch
```

Este preset usa `willowgarage.world` con una pose verificada dentro del contenedor y un perfil de cabeza corregido para que la RGB-D vea un pasillo útil en vez de techo/pared.

7. Lanzar stack completo con HSR oficial.

```
roslaunch vlmap_bringup hsr_gazebo_move_base.launch \
  gui:=true use_rviz:=false use_rosbridge:=false
```

8. Lanzar stack completo con HSR oficial y RViz.

```
roslaunch vlmap_bringup hsr_gazebo_move_base.launch \
  gui:=true use_rviz:=true use_rosbridge:=false
```

9. Lanzar HSR oficial en un mundo concreto.

```
roslaunch vlmap_bringup hsr_gazebo_only.launch \
  world:=/usr/share/gazebo-11/worlds/willowgarage.world \
  gui:=true use_rviz:=true
```

10. Teleoperar el HSR oficial por teclado.

```
roslaunch vlmap_bringup hsr_keyboard_teleop.py
```

Este teleop publica en `/hsrb/command_velocity`, que ahora alimenta el controlador de base del HSR oficial en Gazebo; ya no depende del robot *proxy*.

11. Ver topics relevantes del HSR.

```
rostopic list | egrep 'hsrb|move_base|vlmap|scan|image_rect|camera_info|rectified_points'
```

12. Inspeccionar cámaras y sensores.

```
rostopic hz /hsrb/base_scan
rostopic echo -n 1 /hsrb/head_rgb_sensor/rgb/camera_info
rostopic echo -n 1 /hsrb/head_rgb_sensor/depth_registered/camera_info
```

13. Ver el árbol TF.

```
roslaunch tf view_frames
```

14. Parar Gazebo, RViz y launches activos.

```
pkill -f 'roslaunch vlmap_bringup'
pkill -f gzserver
pkill -f gzclient
pkill -f 'roslaunch rviz rviz'
```

15. Reiniciar limpio el contenedor ROS desde el host.

```
cd /home/mario/tfg/vlmap-semantic-object-search-tfg/docker
docker compose up -d --force-recreate tfg-ros
```

16. Panel RViz de depuración visual.

```
roslaunch vlmap_bringup hsr_gazebo_only.launch gui:=true use_rviz:=true
```

Con ese launch, en RViz conviene comprobar al menos:

- RobotModel: el HSR aparece completo y no colapsa cabeza o brazo.
- TF: existen `map`, `odom`, `base_footprint`, `base_link` y los frames de cámara.
- LaserScan: el topic `/hsrb/base_scan` dibuja lecturas.
- Image: la RGB publica en `/hsrb/head_rgb_sensor/rgb/image_rect_color`.
- PointCloud2: la nube de puntos llega desde `/hsrb/head_rgb_sensor/depth_registered/rectified_po`

15. Comandos de depuración RViz/TF/cámaras.

```
rostopic echo -n 1 /tf
rostopic hz /hsrb/head_rgbd_sensor/rgb/image_rect_color
rostopic hz /hsrb/head_rgbd_sensor/depth_registered/rectified_points
roslaunch rqt_image_view rqt_image_view
```

16. Mundo Gazebo mínimo para probar la GUI.

```
roslaunch gazebo_ros empty_world.launch gui:=true
```

17. Mundo Gazebo simple con geometrías de ejemplo.

```
roslaunch gazebo_ros shapes_world.launch gui:=true
```

18. Mundo Gazebo simple tipo oficina clásica.

```
roslaunch gazebo_ros willowgarage_world.launch gui:=true
```

19. Lanzar el HSR oficial en un mundo Gazebo estándar.

```
roslaunch vlmap_bringup hsr_gazebo_only.launch \
  world:=/usr/share/gazebo-11/worlds/cafe.world \
  gui:=true use_rviz:=true
```

20. Lectura rápida de errores típicos.

- `GetModelState: model [hsrb] does not exist`: suele pasar si un nodo consulta el modelo antes de que Gazebo termine el *spawn*; si el robot aparece después, no es un fallo crítico.
- `ROSInterruptException: ROS shutdown request`: es el apagado normal cuando se mata el launch con `Ctrl+C`; no indica bug.
- `Could not load resource ... torso.std`: eso sí es una referencia errónea de malla. La malla correcta del torso es `torso.stl`; conviene corregir cualquier URDF generado que siga apuntando a `torso.std`.

Índice

1. Objetivo del documento	5
2. Punto de partida actual	6
2.1. Arquitectura vigente	6
2.2. Hechos relevantes del repositorio	6
3. Qué debe migrarse y qué no	6
3.1. Qué sí debe migrarse	6
3.2. Qué no debe migrarse de entrada	6
3.3. Principio de diseño	6
4. Versiones y compatibilidad	7
4.1. Fuentes oficiales	7
4.2. Matriz de decisión	7
4.3. Recomendación técnica	7

5. Arquitectura recomendada	8
5.1. Arquitectura objetivo a corto plazo	8
5.2. Nodos recomendados	8
5.3. Topics / interfaces mínimas	8
6. Estrategia de migración por fases	9
6.1. Fase 0 — preparación y contratos	9
6.2. Fase 1 — ROS en simulación con mapa conocido	9
6.3. Fase 2 — LiDAR simulado y costmap local	9
6.4. Fase 3 — objetivos de habitación y objeto	10
6.5. Fase 4 — integración con HSR	10
7. Decisión sobre ROS 2	10
7.1. Cuándo tiene sentido ir directamente a ROS 2 Humble	10
7.2. Por qué no se recomienda Humble como primer paso	10
8. Riesgos y mitigaciones	11
9. Estrategia de escenas y anotación de habitaciones	11
10. Plan de implementación recomendado (un mes)	12
11. Conclusión	13
12. Bitácora de migración	13

Estado del plan (revisión 23.04.2026)

Este documento se redactó originalmente como una hoja de ruta opcional. En la revisión del 23 de abril de 2026 se decide **activar** la migración como parte del trabajo inmediato del TFG, con plazo de un mes.

Decisiones que motivan el cambio:

- El razonamiento por habitaciones (Fases A–F) y las herramientas de evaluación 2×2 ya están validados en simulación Habitat sobre HSSD. El cuello de botella siguiente es la navegación, lo cual encaja exactamente con el alcance original de este documento.
- Antes de extender el sistema al *nivel 3* (objetos pequeños tipo botellas, tazas, libros), interesa hacerlo directamente sobre el *stack* final (ROS 1 Noetic + simulación con sensores tipo HSR), para no iterar dos veces el mismo trabajo.
- Como referencia conceptual se utiliza explícitamente *MoMa-LLM* (Honerkamp et al., RSS SemRob 2024), del que se adoptan ideas *sin copiar código*: clustering de habitaciones por Voronoi sobre centroides, vocabulario de acciones de alto nivel y evaluación con curvas de eficiencia (AUC-E).

Contexto de host y contenedor. El usuario desarrolla actualmente sobre Ubuntu 22.04 con todo el *stack* encapsulado en un contenedor Docker. Para esta migración el contenedor pasa a ser **Ubuntu 20.04 + ROS 1 Noetic + Gazebo Classic**. El esquema soporta los dos casos:

- **Mientras se trabaja desde Ubuntu 22.04:** el host queda igual, sólo cambia (o se duplica) el contenedor para ejecutar Noetic. No hay que reinstalar nada en el host.
- **Cuando se migre a un equipo con Ubuntu 20.04 nativo:** el mismo contenedor sigue siendo válido. No es obligatorio salirse de Docker, y se recomienda mantenerlo: ya tiene todo instalado y garantiza reproducibilidad para la entrega del TFG y para una eventual ejecución en el laboratorio. El único matiz es que ROS instalado *nativamente* en el host puede dar mejor latencia en aplicaciones de tiempo real con el HSR físico, pero esa optimización no es necesaria para la fase de simulación ni para la entrega.

1. Objetivo del documento

Este documento define una hoja de ruta técnica para introducir ROS en el proyecto actual de búsqueda semántica de objetos. El objetivo no es rehacer el sistema completo, sino desacoplar la navegación de bajo nivel del *stack* manual actual y apoyarse en el ecosistema ROS para:

- usar planificación y control local más robustos;
- aprovechar LiDAR o un */scan* equivalente en simulación;
- preparar el salto a Toyota HSR sin tener que reescribir la capa semántica;
- mantener VLMaps, razonamiento por habitaciones y YOLOE como núcleo de alto nivel.

La tesis de este plan es simple: **la migración a ROS debe afectar sobre todo a la navegación y a la interfaz de robot, no a la semántica**. El valor del proyecto está en cómo decide *dónde* buscar; ROS debe resolver mejor *cómo* llegar.

2. Punto de partida actual

2.1. Arquitectura vigente

Según el documento principal del proyecto (`project_status_and_plan`), el sistema actual ya dispone de:

- razonamiento por habitaciones sobre HSSD;
- VLMaps para memoria semántica de muebles y zonas;
- YOLOE para verificación visual de vocabulario abierto;
- navegación interactiva en Habitat-Sim con un *follower* discreto sobre un mapa 2D derivado del entorno.

El principal cuello de botella ya no está en la selección semántica del objetivo, sino en la navegación de bajo nivel: el robot todavía puede desviarse del camino, rozar paredes, quedarse en la habitación vecina o atascarse en tramos complicados aunque el plan global sea razonable.

2.2. Hechos relevantes del repositorio

- El contenedor actual del proyecto está basado en **Ubuntu 20.04** y CUDA 12.1 (`docker/Dockerfile`).
- El proyecto menciona explícitamente una futura extensión “ROS 1 Noetic + Toyota HSR” en el documento principal.
- El `docker-compose` ya reserva un hueco para un contenedor ROS, pero todavía está deshabilitado.
- El repositorio ya contempla sensor LiDAR a nivel de configuración (`lidar_sensor`, `lidar_fov`, `depth_img_for_lidar_n`), aunque la navegación principal actual no se apoya en un `stack` ROS de navegación.

3. Qué debe migrarse y qué no

3.1. Qué sí debe migrarse

- Navegación 2D de bajo nivel.
- Control local, evasión de obstáculos y recuperaciones.
- Publicación de mapa, odometría, `tf` y sensores virtuales o reales.
- Interfaz con el robot HSR.

3.2. Qué no debe migrarse de entrada

- El razonamiento por habitaciones.
- La selección semántica de objetivos basada en VLMaps.
- La lógica LLM de interpretación y priorización.
- YOLOE como módulo de verificación visual.

3.3. Principio de diseño

La arquitectura objetivo no debe ser “pasar todo a ROS”, sino esta:

Capa	Responsabilidad
VLMaps + LLM + room reasoning	elegir objetivo semántico
ROS navigation stack	llegar al objetivo sin chocar
YOLOE	confirmar visualmente el objeto
HSR / simulador	ejecutar acciones y publicar sensores

4. Versiones y compatibilidad

4.1. Fuentes oficiales

Las decisiones de versiones se apoyan en documentación oficial de ROS:

- ROS 2 Humble: Ubuntu 22.04 como plataforma Tier 1 (<https://docs.ros.org/en/humble/Releases/Release-Humble-Hawksbill.html>).
- ROS 2 Jazzy: Ubuntu 24.04 como plataforma Tier 1 (<https://docs.ros.org/en/jazzy/Releases/Release-Jazzy-Jalisco.html>).
- `ros1_bridge`: Noetic en EOL desde mayo de 2025; soporte pleno en Ubuntu 20.04 y limitaciones serias en 22.04/24.04 (https://docs.ros.org/en/humble/p/ros1_bridge/index.html, https://docs.ros.org/en/humble/How-To-Guides/Using-ros1_bridge-Jammy-upstream.html).

4.2. Matriz de decisión

Opción	Ubuntu	Estado oficial	Ventajas	Desventajas
ROS 1 Noetic	20.04	EOL desde mayo de 2025	Encaja con el Docker actual; buena opción para <i>move_base</i> ; alineado con una posible interfaz HSR histórica	Tecnología sin soporte activo; mala base a largo plazo
ROS 2 Humble	22.04	Soportado; Tier 1 en 22.04	Nav2, ecosistema moderno, mejor apuesta a medio plazo	Requiere cambiar base de entorno; <code>ros1_bridge</code> en 22.04 obliga a build from source y complica convivencia con ROS 1
ROS 2 Jazzy	24.04	Soportado; Tier 1 en 24.04	Mejor horizonte temporal y soporte más largo	Mayor distancia respecto al proyecto actual; no es buena base si se necesita ROS 1 / HSR legacy

4.3. Recomendación técnica

Recomendación principal: migración en dos etapas.

1. **Etapla inmediata de de-risking:** ROS 1 Noetic sobre Ubuntu 20.04 en contenedor aislado, solo para probar si el *navigation stack* resuelve los problemas actuales de seguimiento,

puertas y paredes.

2. **Etapla estratégica futura:** diseñar la interfaz de forma agnóstica para poder sustituir más adelante `move_base` por ROS 2 Humble + Nav2 sin tocar la capa semántica.

La razón de esta recomendación es pragmática:

- el proyecto ya corre sobre Ubuntu 20.04;
- Noetic encaja mejor con el estado actual del entorno;
- si se quiere validar rápido si ROS mejora la navegación, Noetic tiene menor fricción de integración;
- Humble es mejor destino final que Noetic, pero no es la forma más corta de obtener una mejora tangible en navegación desde el estado actual.

Inferencia práctica: si el HSR del laboratorio expone todavía una pila ROS 1 o herramientas equivalentes, esta decisión será todavía más favorable a Noetic en la primera etapa. Esto debe verificarse en el laboratorio antes de fijar la arquitectura definitiva.

5. Arquitectura recomendada

5.1. Arquitectura objetivo a corto plazo

1. El sistema actual decide el objetivo semántico: habitación, celda objetivo o punto de aproximación.
2. Ese objetivo se publica a ROS como goal 2D en frame `/map`.
3. ROS se encarga de:
 - mapa ocupacional,
 - `tf`,
 - coste local,
 - evitación de obstáculos,
 - recuperación,
 - ejecución del camino.
4. Al llegar, el sistema actual recupera el control para la verificación visual con YOLOE.

5.2. Nodos recomendados

Nodo	Responsabilidad
<code>vlmap_semantic_server</code>	carga el VLMap y responde consultas semánticas (habitaciones, categorías, candidatos)
<code>vlmap_task_manager</code>	interpreta la instrucción NL, elige habitación/objeto y pide un goal navegable
<code>habitat_ros_bridge</code>	publica <code>/map</code> , <code>/tf</code> , <code>/odom</code> , <code>/scan</code> , RGB/depth y estado del agente desde Habitat
<code>move_base</code> o <code>Nav2</code>	plan global, control local y recuperación
<code>yoloe_verifier</code>	confirma o rechaza el objeto al llegar

5.3. Topics / interfaces mínimas

- `/map` (`nav_msgs/OccupancyGrid`)

- `/odom` (`nav_msgs/Odometry`)
- `/tf` y `/tf_static`
- `/scan` (`sensor_msgs/LaserScan`)
- `/move_base/goal` en ROS 1 o `/navigate_to_pose` en ROS 2
- `/camera/color/image_raw` y opcionalmente `depth`

6. Estrategia de migración por fases

6.1. Fase 0 — preparación y contratos

Objetivo: fijar interfaces antes de tocar navegación.

- Definir un adaptador único de objetivo: `SemanticGoal = {type, map_pose, room_id, object_class}`.
- Separar en código la capa “decidir goal” de la capa “ejecutar goal”.
- Declarar un backend de navegación intercambiable: `HabitatFollowerBackend` vs `RosNavigationBackend`.

Criterio de salida: el pipeline puede pedir navegación sin saber si la ejecución se hace en Habitat directo o vía ROS.

6.2. Fase 1 — ROS en simulación con mapa conocido

Objetivo: demostrar que ROS navega mejor *sin* añadir todavía SLAM ni hardware real.

- Publicar el mapa de obstáculos actual de `VLMaps` como `OccupancyGrid`.
- Publicar la pose real del agente Habitat como `/odom` y `tf` de referencia.
- Usar mapa fijo conocido; no introducir AMCL ni SLAM todavía.
- Enviar goals desde el selector semántico actual a `move_base`.

Paquetes recomendados (ROS 1 Noetic):

- `map_server`
- `move_base`
- `costmap_2d`
- `base_local_planner` o DWA como primera opción
- `rviz`
- `tf2_ros`

Criterio de salida: comandos como `kitchen`, `bathroom` o `sofa` deben llegar al área correcta con menos choques y menos comportamientos erráticos que el follower actual.

6.3. Fase 2 — LiDAR simulado y costmap local

Objetivo: introducir percepción de obstáculos compatible con el futuro HSR.

- Activar un `/scan` sintético en simulación.
- Primera opción: aprovechar el soporte ya contemplado en configuración (`lidar_sensor`, `lidar_fov=360`).
- Segunda opción: derivar `LaserScan` desde `depth` si el LiDAR nativo no es suficiente.

- Configurar *inflation radius*, footprint y *local costmap*.

Criterio de salida: el robot debe evitar marcos de puerta, paredes y esquinas usando el `local planner`, no una lógica manual dentro de `interactive_object_nav.py`.

6.4. Fase 3 — objetivos de habitación y objeto

Objetivo: mantener la semántica actual y delegar solo la ejecución.

- Habitación: publicar como goal un punto interior de la habitación.
- Objeto: publicar como goal un punto de aproximación seguro.
- Al llegar, ejecutar la lógica actual de verificación con YOLOE.
- Si falla la verificación, pedir el siguiente goal semántico y reenviar al stack ROS.

Criterio de salida: la lógica de reintentos y selección de candidatos sigue en VLMaps; ROS solo ejecuta cada subgoal.

6.5. Fase 4 — integración con HSR

Objetivo: sustituir el bridge Habitat por el robot real.

- Reemplazar `habitat_ros_bridge` por la interfaz del HSR.
- Mantener las mismas entradas: `/map`, `/odom`, `/tf`, `/scan`, RGBD.
- Reutilizar exactamente el mismo `vlmap_task_manager`.

Criterio de salida: el razonamiento de alto nivel no cambia entre simulación y robot.

7. Decisión sobre ROS 2

7.1. Cuándo tiene sentido ir directamente a ROS 2 Humble

Ir directamente a ROS 2 Humble tendría sentido si se cumplen **dos** condiciones:

1. no se necesita integrar a corto plazo con una pila ROS 1 del HSR;
2. se acepta migrar el entorno base de Ubuntu 20.04 a 22.04.

En ese caso, la arquitectura equivalente sería:

- `nav2_map_server`
- `planner_server`
- `controller_server`
- `bt_navigator`
- `behavior_server`
- `rviz2`

7.2. Por qué no se recomienda Humble como primer paso

No se recomienda como primer paso porque:

- el proyecto actual ya está asentado sobre Ubuntu 20.04;
- el experimento que interesa ahora es “¿mejora ROS la navegación?”, no “¿puedo rehacer hoy toda la infraestructura?”;

- si además se necesitara convivir con ROS 1, el propio `ros1_bridge` en Jammy obliga a una configuración más delicada y a compilación desde código fuente.

8. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Meter ROS demasiado pronto y bloquear el proyecto	Sobrecoste de integración	Migración por fases; primero simulación con mapa conocido
Noetic está en EOL	Deuda técnica	Usarlo solo como etapa táctica; diseñar interfaces portables a ROS 2
Goal semántico incorrecto	ROS llega bien al sitio equivocado	Mantener y endurecer la selección semántica fuera de ROS
Exceso de scope con HSR real	Retraso del roadmap	Validar primero en simulación; HSR solo cuando el backend ROS funcione

9. Estrategia de escenas y anotación de habitaciones

La migración renuncia a depender exclusivamente de HSSD. El nuevo stack trabaja con escenas más naturales para ROS 1 Noetic + Gazebo:

- **Escenas Gazebo de calidad equivalente o superior:** *house worlds* oficiales del HSR, `aws-robomaker-small-house-world`, escenarios académicos publicados (TIAGo, PAL Robotics, etc.). Permiten montar el sistema entero (HSR simulado, sensores, costmaps) sin tener que convertir HSSD a SDF.
- **Opción intermedia: iGibson** (15 apartamentos basados en escaneos reales) si se necesita comparabilidad con la literatura de *semantic search*; es el conjunto que utiliza MoMa-LLM, lo que facilita citar resultados.

Etiquetado de habitaciones por centroides (estilo MoMa-LLM)

HSSD venía con polígonos por habitación (`semantic_config.json`). Las escenas Gazebo y, sobre todo, el laboratorio físico no los traen; anotarlos a mano es costoso y poco escalable. Se adopta el esquema de Voronoi sobre centroides, inspirado en MoMa-LLM:

1. El usuario marca *un punto* (centroide) por habitación, junto con su nombre semántico (`kitchen`, `bathroom`, ...). Esto puede hacerse con un click sobre el mapa de ocupación en RViz.
2. Cada celda navegable del mapa se asigna a su centroide más cercano: partición de Voronoi.
3. Esa partición sustituye al sistema de polígonos. La capa `room_provider` (ya existente) se reescribe con esta lógica, manteniendo su interfaz pública. Todas las métricas room-aware (CFR, CT2R, Rooms Before Success, Final Room Accuracy) siguen siendo calculables sin modificarlas.
4. El mismo esquema funciona en HSSD, en escenas Gazebo y en el laboratorio real: la única diferencia es de dónde proviene la lista de centroides.

Coste estimado de anotación: 5–10 minutos por escena, frente a las varias horas que requiere etiquetar polígonos. Coste de implementación del labeler centroides→Voronoi: aproximadamente una sesión.

10. Plan de implementación recomendado (un mes)

El plazo asumido es **un mes natural** desde el 23 de abril de 2026, con apoyo de Codex y Claude Code para acelerar tareas de plumbing. Los cuatro *sprints* están dimensionados para encajar en esa ventana.

Sprint 1 — contenedor ROS 1 Noetic + escena base (semana 1)

1. Crear o adaptar el `Dockerfile` con Ubuntu 20.04, ROS 1 Noetic `ros-noetic-desktop-full`, Gazebo Classic, `move_base`, `map_server`, `costmap_2d`, `tf2_ros`, `rviz`, drivers Python (`rospy`, `cv_bridge`).
2. Levantar una *house world* de Gazebo con un robot móvil simulado (HSR simulado si está accesible; si no, TurtleBot3 o Husky como *stand-in* mientras se prepara HSR).
3. Hacer un *smoke test*: enviar un goal fijo a `move_base` y comprobar en RViz que el robot llega.

Criterio de salida: contenedor reproducible y un goal manual ejecutado correctamente.

Sprint 2 — VLMMap, room labeler y goals semánticos (semana 2)

1. Implementar el recolector ROS de RGB+depth+pose (suscriptor a los topics correspondientes) que produzca el dataset que come el *builder* actual de VLMMap. Reutilizar el *builder* sin cambios estructurales.
2. Implementar el labeler centroide→Voronoi y reemplazar `room_provider` con la nueva fuente.
3. Escribir el nodo `vmap_task_manager`: recibe la instrucción en lenguaje natural, ejecuta el razonamiento por habitaciones existente y publica goals (`move_base_msgs/MoveBaseGoal`).

Criterio de salida: comandos como `kitchen`, `bathroom` o `sofa` llegan al área correcta con el stack ROS, sobre la nueva escena.

Sprint 3 — nivel 3, YOLOE y harness de evaluación (semana 3)

1. Conectar el verificador YOLOE al pipeline ROS: nodo que se suscriba a la cámara RGB del robot y exponga un servicio de verificación.
2. Activar la capa de *surrogate furniture* (Phase G ya existente) para queries de objetos pequeños.
3. Re-cablear el harness de evaluación 2×2 para ejecutarse contra el endpoint ROS en lugar del de Habitat. Los manifests y métricas se reutilizan tal cual (formato `[eval-summary]` ya estabilizado).

Criterio de salida: batería de queries ejecutable extremo a extremo sobre Gazebo+ROS, con manifests comparables a los de Habitat.

Sprint 4 — consolidación, métricas y memoria (semana 4)

1. Lanzar la batería completa sobre 2–3 escenas, recoger métricas y comparar con los resultados HSSD/Habitat (validación de portabilidad del razonamiento).

2. Si hay disponibilidad del HSR físico: *smoke test* de los nodos sobre el robot real, sin tocar la capa semántica.
3. Redacción de los capítulos de la memoria correspondientes: migración, labeler de habitaciones, resultados Gazebo+ROS y, opcionalmente, demostración en HSR.

Criterio de salida: memoria del TFG con resultados en dos *stacks* (Habitat para el room-aware, ROS+Gazebo para nivel 3) y una demostración (real o en simulación HSR-sim) lista para defensa.

Riesgos del calendario. El mes es ajustado. Los riesgos principales son: (i) tiempo perdido en configurar HSR-sim si el paquete no está mantenido para Noetic; (ii) debug de `move_base` (*tuning* de `costmap`, `inflation radius`); (iii) bugs en el bridge YOLOE↔ROS. Mitigación: si el HSR-sim no está disponible en la primera semana, el sprint 1 usa TurtleBot3/Husky como *stand-in* y el sprint 4 sustituye HSR-sim por “Gazebo + robot genérico”, sin que la contribución del TFG dependa del HSR.

11. Conclusión

Sí merece la pena preparar la migración a ROS, pero **solo para la parte de navegación y sensorización**. Ese cambio tiene sentido técnico porque el problema actual está concentrado en el seguimiento y la evasión local, no en la semántica.

La ruta recomendada es:

1. **Noetic + Ubuntu 20.04** como experimento táctico para validar si `move_base` y *costmaps* resuelven el problema actual.
2. Diseñar la integración para que después pueda migrarse a **ROS 2 Humble + Nav2** si el proyecto evoluciona hacia una base de software mantenible a medio plazo.

En otras palabras: **migrar la navegación, no reescribir el proyecto.**

12. Bitácora de migración

24 de abril de 2026 — Sprint 1 scaffolding completado en paralelo

Mientras la fase de orquestador estresada terminaba sobre HSSD, se abrió una rama paralela de trabajo para dejar la infraestructura ROS lista antes de empezar Sprint 2. Esta sesión cubre Sprint 0 (contratos) y Sprint 1 (contenedor + workspace) del plan; ningún componente de Sprint 2 en adelante se ha tocado.

Decisión de arquitectura: dos contenedores con responsabilidades estrictas. El contenedor existente `tfg-sim` mantiene Habitat-Sim, CLIP, YOLOE y la lógica de razonamiento; el nuevo contenedor `tfg-ros` aloja sólo ROS 1 Noetic, Gazebo Classic y el *stack* de navegación. Comunicación entre ambos por `rosbridge` sobre WebSocket en el puerto 9090, en línea con la práctica habitual de *stacks* multi-entorno (MoMa-LLM utiliza un patrón equivalente). Esto evita instalar ROS dentro del entorno `conda` y evita instalar `torch`/CLIP en el sistema Python 3.8 que trae Noetic.

Entregables de la sesión:

- `docker/Dockerfile.ros` basado en `osrf/ros:noetic-desktop-full-focal` con Gazebo, `move_base`, `costmap_2d`, `base_local_planner`, `rviz`, `rosbridge_suite` y dependencias Python mínimas (`numpy`, `opencv-python-headless`, `roslibpy`, `pyyaml`).

- `docker/entrypoint_ros.sh` con tres modos (`shell`, `core`, `launch:<file>`) controlados por la variable `ROS_MODE`.
- Servicio `tfg-ros` activado en `docker/docker-compose.yml`: monta `ros_ws/` en `/ros_ws`, ... en `/workspace`, expone el puerto 9090, integra la red `tfg-net` ya existente y *no* declara `depends_on: tfg-sim` para que ambos contenedores puedan operar de forma independiente.
- Workspace catkin nuevo en `ros_ws/src/` con seis paquetes: `vlmap_msgs` (mensajes y servicios: `SemanticGoal.msg`, `QueryRoom.srv`), `vlmap_semantic_server`, `vlmap_task_manager`, `habitat_ros_bridge`, `yoloe_verifier` y `vlmap_bringsup` (con `minimal.launch`, `gazebo_house.launch`, `minimal.rviz` y `move_base_params.yaml` por defecto). Cada nodo es un *stub* que se registra en `rosout` y publica un `/vlmap/heartbeat` para que el smoke test verifique el grafo. Cada `__init__.py` documenta los módulos Python existentes que envolverá Sprint 2.
- Capa de contratos `src/tfg_nav_contracts/`: dataclass `SemanticGoal` (espejo del mensaje ROS) y la abstracción `NavigationBackend` con dos subclases *stub* (`HabitatFollowerBackend` y `RosNavigationBackend`) que levantan `NotImplementedError` con un TODO Sprint 2 explícito.
- Tests unitarios sin Habitat ni ROS en `tests/test_semantic_goal_contract.py` (5 tests pasados) que verifican el round-trip JSON del dataclass y que ambos backends cumplen la interfaz abstracta.
- `ros_ws/README.md` con la guía de arranque rápido, descripción de la arquitectura objetivo y mapa de reutilización de módulos para Sprint 2.

Restricciones respetadas: ningún proceso del contenedor `tfg-sim` se ha matado (la fase orquestador estresada estaba corriendo); el submódulo `third_party/vlmaps` no se ha modificado (mantiene los cambios locales del usuario en `search_state.py` y `executor.py`); ningún commit se ha hecho desde el asistente.

Pendiente para mañana (Sprint 2, semana 1):

1. `docker compose build tfg-ros` y verificación `catkin build` dentro del contenedor.
2. Decidir el robot simulado de partida: HSR-sim si está disponible para Noetic, o TurtleBot3/Husky como *stand-in* si no.
3. Decidir la escena Gazebo de partida (recomendado: `aws-robomaker-small-house-world` o un *house world* oficial del HSR).
4. Implementar el labeler centroide → Voronoi en `vlmap_semantic_server`.
5. Implementar el recolector RGB+depth+pose en `habitat_ros_bridge` para reconstruir el VLMa sobre la nueva escena.

28 de abril de 2026 — Sprint 2 parcial validado dentro de Docker

Se completa el primer bloque funcional de Sprint 2 sin tocar el pipeline Habitat/HSSD ya estabilizado. La restricción de arquitectura se mantiene: `tfg-sim` conserva VLMa, HSSD, YOLOE y la evaluación existente; `tfg-ros` asume exclusivamente ROS 1 Noetic, topics, servicios y la futura navegación con `move_base`. El objetivo de esta sesión no ha sido “portar Habitat a ROS”, sino fijar una interfaz limpia entre ambos contenedores para que más adelante Gazebo, HSR-sim o el HSR físico puedan sustituir el origen de mapa/pose sin reescribir la capa semántica.

Infraestructura validada de verdad:

- La imagen `tfg-ros` se construye correctamente desde `docker/Dockerfile.ros`.

- El servicio `tfg-ros` arranca con `docker compose up -d` manteniendo independencia respecto de `tfg-sim`.
- El workspace `/ros_ws` se compila dentro del contenedor tras fijar `catkin config -extend /opt/ros/noetic`. Resultado: **7 paquetes compilados** en la primera validación y **6 paquetes compilados** tras la introducción del nuevo servicio de resolución de habitaciones.
- El *smoke test* con `roslaunch vlmap_bringup minimal.launch` levanta `rosbridge`, `rosapi`, `vlmap_semantic_server`, `vlmap_task_manager`, `habitat_ros_bridge` y `yoloe_verifier`, todos visibles vía `/vlmap/heartbeat`.

Nuevos bloques implementados:

- `vlmap_semantic_server/centroid_voronoi.py`: primer *room provider* agnóstico a Habitat, basado en centroides etiquetados manualmente y partición de Voronoi sobre celdas libres. Expone `RoomCentroid`, `build_voronoi_room_map` y `VoronoiRoomProvider`. Este es el reemplazo previsto del sistema de polígonos HSSD para Gazebo y laboratorio.
- Servicio `/vlmap/query_room`: ya no es un *stub*; devuelve un ranking de habitaciones para una categoría libre (por ejemplo `laptop`) reutilizando `vlmaps.utils.room_priors` cuando el submódulo está montado en `/workspace`.
- Servicio nuevo `/vlmap/resolve_room` (`vlmap_msgs/ResolveRoom.srv`): traduce una habitación semántica a un punto 2D en frame `/map`, usando el centroide Voronoi y los metadatos del mapa (resolución y origen).
- `habitat_ros_bridge/file_bridge.py`: helpers *file-backed* para cargar mapas de ocupación (`.npy/.json`) y una pose 2D externa. El nodo `habitat_ros_bridge` publica ya `/map`, `/odom` y `tf: map→odom→base_link` sin depender todavía del runtime interno de Habitat. Esto fija una interfaz estable que mañana puede alimentarse desde Gazebo o desde el HSR real.
- `vlmap_task_manager/parsing.py` y actualización del nodo `vlmap_task_manager`: ya acepta instrucciones sencillas (`find laptop`, `search for mug`, `find the laptop in the office`), consulta `/vlmap/query_room` y `/vlmap/resolve_room`, y publica tanto `/vlmap/semantic_goal` como `/move_base_simple/goal`. La tubería *instruction* → *room ranking* → *room centroid* → *SemanticGoal* → *ROS nav goal* queda operativa.

Validación reproducible realizada en esta sesión:

- Tests offline de la migración: `tests/test_ros_voronoi_room_labeler.py` y `tests/test_ros_bridge_utils.py` con **7 tests pasados**.
- Prueba sintética de `/vlmap/query_room`: para dos habitaciones (`kitchen`, `office`), la consulta `laptop` devuelve `office` por encima de `kitchen`.
- Prueba end-to-end dentro de `tfg-ros`: con un mapa 5×5, dos centroides y una pose inicial serializada, el sistema publica correctamente:
 - `/map` como `nav_msgs/OccupancyGrid`,
 - `/odom` con la pose esperada,
 - `/move_base_simple/goal` tras publicar `find laptop` en `/vlmap/instruction`.

Lectura técnica del estado: la migración sigue bien diferenciada. La parte HSSD no se ha modificado ni duplicado; lo que se ha construido es una capa ROS autónoma, pensada para consumir mapa/pose/objetivos semánticos. Eso reduce el riesgo de contaminación experimental y permite seguir usando HSSD como banco de validación mientras se construye el backend ROS/Gazebo/HSR.

Siguiente paso recomendado en el cierre de esta sesión:

1. cablear el backend `RosNavigationBackend` con `rosbridge` para que el contenedor semántico pueda mandar goals directamente al contenedor ROS;
2. sustituir el origen *file-backed* del bridge por un publicador vivo desde `tfg-sim` o por Gazebo;
3. una vez estable, conectar `move_base` a una escena Gazebo realista y dejar HSR-sim como siguiente backend físico-simulativo.

28 de abril de 2026 — Backend desacoplado ROS validado

En una iteración posterior del mismo día se completa precisamente el punto que quedaba pendiente en la entrada anterior: el contrato Python `SemanticGoal` $\rightarrow$ `NavigationBackend` deja de ser un mero *placeholder* y pasa a disponer de un backend ROS funcional, independiente de `rospy` y preparado para ejecutarse desde `tfg-sim` sin contaminar el entorno Noetic.

Implementación cerrada en esta iteración:

- `src/tfg_nav_contracts/navigation_backend.py`: la clase `RosNavigationBackend` ya no levanta `NotImplementedError`. Ahora:
 - abre conexión con `rosbridge` vía `roslibpy`;
 - publica un espejo semántico en `/vlmap/semantic_goal`;
 - publica el objetivo 2D correspondiente en `/move_base_simple/goal`;
 - soporta cancelación vía `/move_base/cancel`;
 - acepta resultados tanto desde `/move_base/result` como desde un topic JSON intermedio, útil durante la migración.
- Se mantiene una decisión deliberada de diseño: **un único goal activo a la vez**. Esta restricción coincide con la política secuencial actual del sistema semántico y simplifica mucho la convivencia entre `tfg-sim`, Gazebo, HSR-sim y, más adelante, el HSR físico.
- `tests/test_semantic_goal_contract.py` se amplía para cubrir el backend ROS con clientes y topics falsos, sin necesidad de levantar `roscore`. Queda validado que:
 - se publican correctamente `SemanticGoal` y `PoseStamped`;
 - la cancelación genera el `GoalID` esperado;
 - los resultados JSON y los resultados compatibles con `move_base` se traducen al contrato `NavigationResult`.
- `ros_ws/README.md` se actualiza para reflejar que la línea ROS ya no es sólo *scaffolding*: existe un baseline puenteable entre contenedores, con responsabilidades bien separadas entre `tfg-sim` y `tfg-ros`.

Validación ejecutada:

- `pytest tests/test_semantic_goal_contract.py tests/test_ros_bridge_utils.py tests/test_ros`
-v $\rightarrow$ **15 tests pasados**.
- `py_compile` del backend Python $\rightarrow$ correcto.

Conclusión técnica: a partir de este punto la separación entre contenedores queda bien fijada. `tfg-sim` puede seguir siendo el banco de validación HSSD/Habitat, mientras que `tfg-ros` pasa a ser el backend de navegación sustituible. La futura transición a Gazebo o al HSR físico no exigirá portar la lógica semántica, sino sólo alimentar mejor el bridge ROS y conectar un controlador real de navegación.

Plan operativo para integrar Toyota HSR en Gazebo

La integración del HSR no debe plantearse como “meter el robot entero” en un único salto. El criterio de ingeniería es mantener estable el contrato ya validado:

`SemanticGoal → rosbridge → move_base/HSR → navigation_result`

Lo que cambia por debajo es el backend físico-simulado. Primero se reemplaza el robot diferencial mínimo por HSR-sim; después se conectan cámaras, sensores y colisiones; finalmente se añade manipulación.

Fase 1 — disponibilidad real del stack HSR.

El diagnóstico inicial confirma que no conviene depender de paquetes instalados en el host Ubuntu 22.04. La integración se mantiene dentro del contenedor `tfg-ros` (Ubuntu 20.04 + ROS 1 Noetic). Los paquetes públicos que ya se incorporan localmente al workspace son:

- `hsr_meshes`: mallas visuales y de colisión del Toyota HSR.
- `hsr_description`: URDF/Xacro oficial público, incluyendo enlaces, articulaciones, sensores y geometría del robot.
- `vlmap_bringup`: lanzadores propios para aislar la integración HSR del stack Habitat/HSSD.

La descarga se reproduce mediante `tools/fetch_hsr_official_models.py`. Los repositorios externos se guardan bajo `ros_ws/src/external/` y se excluyen de Git: el proyecto versiona el código de integración, no vendorea assets grandes de terceros.

Para ROS 1 Noetic ha sido necesario aplicar parches de compatibilidad Xacro: `macro` pasa a `xacro:macro`, `insert_block` pasa a `xacro:insert_block` y se corrige el nombre de transmisiones generado por el Xacro. Este último punto resolvió el síntoma observado en Gazebo de que al robot le faltaba la cabeza: los bloques de origen no se estaban expandiendo y varios enlaces quedaban colapsados en el origen.

Quedan como dependencias opcionales, no bloqueantes para el cierre inicial, `hsrb_gazebo_launch`, mundos TMC/WRS y controladores oficiales de base, cabeza, torso, brazo y gripper.

Fase 2 — sustituir el robot stand-in por HSR-sim.

El launch actual `gazebo_move_base.launch` ya valida Gazebo, `/odom`, `/scan`, `move_base`, `rosbridge` y `/vlmap/navigation_result` con un robot diferencial mínimo. La versión HSR será un launch paralelo, no una reescritura destructiva:

```
ros_ws/src/vlmap_bringup/launch/hsr_gazebo_move_base.launch
ros_ws/src/vlmap_bringup/config/hsr_move_base_params.yaml
ros_ws/src/vlmap_bringup/config/hsr_topics.yaml
```

La sustitución ya implementada para el perfil oficial es:

```
<param name="robot_description"
  command="$(find xacro)/xacro
    '$(find hsr_description)/robots/hsrb4s.urdf.xacro'"/>
<node pkg="gazebo_ros" type="spawn_model"
  args="-urdf -param robot_description -model hsrb"/>
```

El estado actual queda dividido en dos perfiles:

- `hsr_proxy_gazebo_move_base.launch`: robot ligero con geometría proxy, navegación diferencial y contrato `SemanticGoal → move_base` ya validable.
- `hsr_gazebo_move_base.launch`: modelo HSR oficial público, con mallas reales, árbol TF y sensores. Es el perfil de fidelidad para RViz, cámaras, colisiones y transición posterior a

robot físico.

La prueba mínima de esta fase ya no es sólo conceptual: el HSR oficial aparece en Gazebo como `hsrb`, publica `/tf` y `/joint_states`, y la cabeza queda en su posición correcta tras los parches Xacro. El perfil proxy se mantiene como fallback navegable mientras no estén conectados los controladores oficiales de base.

Fase 3 — frames, odometría y base móvil.

El objetivo mínimo de navegación es:

`map → odom → base_footprint/base_link`

Los nombres reales alineados con el HSR oficial se fijan en `hsr_topics.yaml`:

- comando de base para el perfil proxy: `/cmd_vel`;
- laser de navegación: `/hsrb/base_scan`;
- odometría Gazebo del HSR: `/hsrb/odom_ground_truth`;
- frame base: `base_footprint` o `base_link`;
- TF estático temporal `map → odom → base_footprint` cuando se lanza el HSR oficial sin controladores de base completos.

La configuración HSR de `move_base` ajustará:

```
global_costmap:
  robot_base_frame: base_footprint
```

```
local_costmap:
  robot_base_frame: base_footprint
  observation_sources: scan
  scan:
    topic: /hsrb/base_scan
```

No se cambia el contrato semántico. Se mantiene el mismo flujo: `RosNavigationBackend` publica `goals` y espera `/vlmap/navigation_result`.

Fase 4 — cámaras, sensores y verificación visual.

Para object search hacen falta, como mínimo:

- cámara RGB para YOLOE;
- depth registrado para geometría, orientación o pseudo-laser;
- `camera_info` para proyectar detecciones si hace falta;
- `/tf` coherente entre cámara, cabeza y base;
- odometría y laser para navegación local.

Topics esperados a validar, sujetos al stack HSR real:

- `/hsrb/head_rgbd_sensor/rgb/image_rect_color`;
- `/hsrb/head_rgbd_sensor/depth_registered/image_rect_raw`;
- `/hsrb/head_rgbd_sensor/rgb/camera_info`;
- `/hsrb/head_rgbd_sensor/depth_registered/camera_info`;
- `/hsrb/head_rgbd_sensor/depth_registered/rectified_points`;

- `/hsrb/base_scan`;
- `/hsrb/odom_ground_truth`;
- `/tf`;
- `/joint_states`.

La primera integración de percepción no debe mover YOLOE entero a `tfg-ros`. La opción conservadora es que `yolo_verifier` exponga una interfaz ROS y delegue la inferencia pesada al stack semántico o a un servicio dedicado, manteniendo torch/YOLOE fuera del entorno Noetic cuando sea conveniente.

Fase 5 — colisiones y footprint.

Las colisiones del HSR no se deben inventar manualmente. La fuente de verdad debe ser el URDF/Xacro oficial, porque incluye:

- mallas de colisión;
- inercias;
- límites articulares;
- poses de sensores;
- plugins Gazebo;
- interfaces de control.

Sólo se tocarán parámetros 2D si `move_base` interpreta mal el cuerpo del robot:

- `footprint`;
- `inflation_radius`;
- `robot_base_frame`;
- capas de costmap y fuentes de observación.

La prueba de aceptación de esta fase es que el HSR pueda llegar a un goal 2D en Gazebo sin que `move_base` aborte por colisión falsa, y que `/vlmap/navigation_result` preserve `actual_room`.

Fase 6 — manipulación y brazo.

La manipulación queda explícitamente fuera del primer cierre de navegación. El orden correcto es:

1. HSR en Gazebo con base móvil funcional;
2. sensores RGB-D y cabeza orientable;
3. verificación visual con YOLOE desde topics ROS;
4. `MoveIt` o interfaz HSR equivalente para brazo y gripper;
5. tareas *pick-and-place* y OVMM.

Esto permite cerrar el TFG con object search y navegación semántica, y dejar manipulación como extensión posterior o como conexión natural con la línea de Pau.

Criterio de éxito inicial.

El primer milestone HSR se considera cumplido cuando:

- HSR-sim aparece en Gazebo;
- publica `/tf`, `/odom`, `/scan` y cámara RGB-D;

- acepta comandos de base;
- `move_base` alcanza un goal 2D;
- `RosNavigationBackend.wait_result()` recibe `success=True` a través de `/vlmap/navigation_result`.

Hasta entonces, el robot proxy se mantiene como *stand-in* para validar la infraestructura ROS sin bloquearse por dependencias concretas del HSR.

Estado actualizado y plan de ejecución propuesto

La línea ROS ya no está en fase de prueba aislada. A estas alturas existe una base funcional sobre `tfg-ros` con:

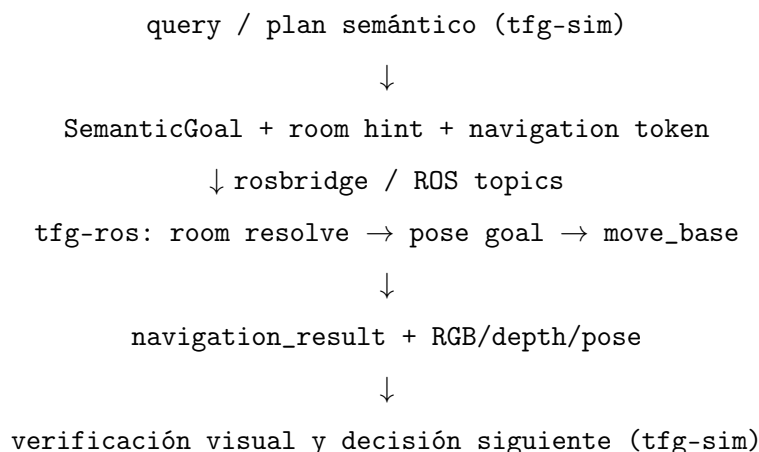
- HSR oficial en Gazebo;
- RViz con sensores principales;
- `move_base`, `/odom`, `/tf` y `/vlmap/navigation_result`;
- `rosbridge` y contrato semántico básico;
- separación real entre `tfg-sim` y `tfg-ros`.

La decisión correcta a partir de este punto no es “seguir añadiendo piezas”, sino ejecutar una migración por capas con criterios de aceptación estrictos. El objetivo es que el *backend* ROS quede listo para recibir el cerebro semántico de `tfg-sim` sin reescribir la lógica de object search ni degradar el banco de validación HSSD/Habitat.

Arquitectura objetivo. El reparto de responsabilidades queda fijado así:

- `tfg-sim`: razonamiento semántico, selección de habitación, heatmaps, verificación visual pesada, evaluación HSSD y lógica de búsqueda.
- `tfg-ros`: ROS 1 Noetic, Gazebo, HSR simulado o real, sensores, `move_base`, `tf`, odometría y ejecución física del movimiento.
- **Contrato entre contenedores**: goals de navegación, room hints, odometría/pose, RGB/depth, timestamps y resultados de navegación.

La arquitectura final que se quiere estabilizar es:



Principio de migración. No se debe migrar la inteligencia a ROS. ROS sólo debe ejecutar navegación y servir sensores. Toda lógica que hoy ya funciona o se valida en `Habitat + HSSD` debe seguir viviendo fuera del contenedor ROS. El trabajo de migración consiste en sustituir el *backend* físico-simulado, no en duplicar la capa semántica.

Fase 1 — cerrar la navegación base del HSR oficial. Objetivo: dejar el HSR oficial como backend de navegación único en Gazebo, eliminando cualquier dependencia operativa del robot proxy.

Trabajo:

- fijar un mundo estático mínimo y reproducible para pruebas (`minimal_walls.world` o equivalente);
- estabilizar `move_base` con footprint, inflation, tolerancias y goal acceptance razonables para el HSR;
- dejar teleoperación y goals 2D de RViz funcionando de forma predecible;
- validar que `/odom`, `/tf`, `/hsrb/base_scan` y `/hsrb/command_velocity` forman una cadena coherente.

Criterio de aceptación:

- el HSR arranca en Gazebo sin colapsar;
- acepta teleop y 2D Nav Goal;
- `move_base` llega a goals simples sin movimientos erráticos;
- el mismo launch funciona de forma repetible tras reinicios del contenedor.

Fase 2 — fijar el contrato estable entre ROS y semántica. Objetivo: convertir el contrato ROS actual en la interfaz definitiva que consumirá `tfg-sim`.

Trabajo:

- consolidar `SemanticGoal`, `/move_base_simple/goal` y `/vlmap/navigation_result` como plano mínimo de navegación;
- añadir a ese contrato un identificador estable de petición (`navigation_token`) y room hint explícito;
- dejar documentado qué topics se consideran públicos y cuáles son internos del bringup;
- evitar que `tfg-sim` dependa de detalles de Gazebo, nombres de plugins o controladores concretos.

Criterio de aceptación:

- desde fuera de `tfg-ros` se puede pedir una navegación sin saber nada del HSR, salvo el contrato ROS expuesto;
- el resultado de navegación vuelve con token, éxito/fallo y habitación efectiva.

Fase 3 — sustituir el backend file-backed por uno vivo. Objetivo: dejar atrás la dependencia del bridge file-backed como ruta principal de ejecución.

Trabajo:

- mantener `habitat_ros_bridge` sólo como herramienta de compatibilidad o prueba;
- hacer que `vlmap_semantic_server` pueda recibir centroids, room priors o mapas desde una fuente viva procedente de `tfg-sim`;
- decidir si esa alimentación viva entra por `rosbridge`, servicio ROS o topic propio;
- separar claramente el modo *demo local* del modo *integrado con tfg-sim*.

Criterio de aceptación:

- el room provider de ROS ya no depende de JSON estáticos para el flujo principal;
- `tfg-sim` puede actualizar o publicar el contexto semántico sin reiniciar `tfg-ros`.

Fase 4 — conectar percepción ROS con el verificador visual. **Objetivo:** que el backend ROS entregue a la capa semántica todo lo necesario para verificar objetivos desde cámaras reales/simuladas del HSR.

Trabajo:

- consolidar topics RGB, depth, `camera_info` y pose de cámara;
- dejar un nodo o puente que capture frames del HSR y los exponga a `tfg-sim` o a un servicio de verificación;
- mantener la inferencia pesada fuera de Noetic siempre que no sea estrictamente necesario moverla;
- definir cómo se devuelve el resultado visual: detectado/no detectado, score, pose estimada, timestamp.

Criterio de aceptación:

- una orden de verificación puede consumir la RGB del HSR y devolver un resultado estructurado reutilizable por el buscador;
- la cadena `map` → `odom` → `base` → cámara está validada.

Fase 5 — cerrar el bucle end-to-end entre contenedores. **Objetivo:** ejecutar el mismo patrón lógico del object search, pero con backend ROS/Gazebo.

Trabajo:

- `tfg-sim` emite plan semántico y room hint;
- `tfg-ros` resuelve habitación, navega y devuelve `navigation_result`;
- `tfg-sim` pide verificación visual;
- si falla, decide la siguiente pose/room retry y repite.

Criterio de aceptación:

- existe una demo reproducible donde ambos contenedores colaboran sin tocar manualmente topics internos;
- el backend ROS es sustituible: Gazebo hoy, HSR físico mañana.

Fase 6 — preparar la transición a HSR físico. **Objetivo:** que el salto a hardware real sea un cambio de bringup, no de arquitectura.

Trabajo:

- conservar nombres de topics/frames compatibles con el HSR real siempre que sea viable;
- encapsular diferencias Gazebo vs físico en launch/config;
- no acoplar el cerebro semántico a odometría ground-truth ni a atajos de simulación;
- dejar una tabla de equivalencias entre sensores/frames de Gazebo y del robot real.

Criterio de aceptación:

- el cambio principal para pasar a hardware real es reemplazar el bringup y revisar topics, no reescribir `tfg-sim`.

Orden de implementación recomendado. Si el trabajo semántico en Habitat lo sigue desarrollando otro agente, el orden práctico para esta línea ROS es:

1. navegación robusta del HSR oficial en un mundo simple;
2. contrato definitivo entre contenedores;
3. fuente viva para room provider / mapa semántico;
4. puente de percepción ROS $\leftrightarrow$ verificador;
5. demo end-to-end entre `tfg-sim` y `tfg-ros`;
6. endurecimiento para HSR físico.

Qué no se debe hacer.

- No portar a ROS la lógica completa de object search.
- No romper el flujo HSSD/Habitat, que sigue siendo el banco de verdad.
- No mezclar manipulación con navegación hasta cerrar el bucle semántico.
- No introducir dependencias del HSR físico dentro del núcleo semántico.

Milestone útil para pedir visto bueno. Antes de implementar nuevas capas, el milestone que debe aprobarse es este:

“Queremos que `tfg-ros` quede como backend navegable y sensorizado del HSR oficial, y que `tfg-sim` pueda hablar con él mediante un contrato estable de navegación y percepción, sin mover la inteligencia principal fuera de Habitat/HSSD”.

Estado implementado tras aprobar este plan. La hoja de ruta anterior ya no es sólo propuesta. Quedan implementadas y validadas las siguientes piezas del contrato:

- **Contexto semántico dinámico:** `vlmap_semantic_server` ya acepta un topic JSON `/vlmap/room_context` (`std_msgs/String`) con habitaciones, poses 2D y priors por categoría. Si ese contexto existe, `/vlmap/query_room` y `/vlmap/resolve_room` lo usan antes de caer al modo Voronoi/file-backed.
- **Publicador auxiliar de contexto:** se añade `publish_room_context.py` para inyectar un JSON local en ROS sin depender todavía de `tfg-sim`.
- **Contrato de verificación visual:** se añade `vlmap_msgs/VerifyObject.srv` y el nodo `yoloe_verifier` deja de ser sólo un heartbeat. Ahora expone el servicio `/vlmap/verify_object`, publica requests JSON en `/vlmap/verification/request` y espera resultados JSON en `/vlmap/verification/result`.
- **Worker mock de verificación:** se añade `mock_verify_worker.py` para probar el bucle de verificación sin mover todavía la inferencia pesada a Noetic ni tocar el contenedor semántico.
- **Cierre del bucle de navegación desde instrucciones:** el `vlmap_task_manager` genera ya `navigation_token` al emitir un `SemanticGoal` desde `/vlmap/instruction`. Además, el bridge `vlmap_navigation_result` amortigua resultados terminales huérfanos de `move_base` para evitar carreras cuando el backend responde demasiado rápido.

Validación ejecutada:

- `pytest` offline sobre contrato, Voronoi, contexto dinámico y bridge de verificación visual: **21 tests pasados.**

- `py_compile` correcto de los scripts nuevos.
- `catkin build` correcto de `vlmap_msgs`, `vlmap_semantic_server`, `yoloe_verifier`, `vlmap_task_manager`, `habitat_ros_bridge` y `vlmap_bringup`.
- Smoke test ROS real dentro de `tfg-ros`:
 - `/vlmap/query_room` devuelve `office > kitchen` para `laptop` desde contexto dinámico.
 - `/vlmap/verify_object` devuelve `success=True`, `found=True` contra el worker mock.
 - `find laptop` sobre `/vlmap/instruction` produce un `/vlmap/navigation_result` JSON correcto, con `token`, `actual_room=office` y `success=True`.

Bitácora de migración

2026-05-13 — Visualización del VLMap desde RViz. Ampliamos `vlmap_semantic_server` para que cada llamada a `/vlmap/query_room` publique además dos topics de visualización:

- `/vlmap/heatmap` (`nav_msgs/OccupancyGrid`, `latched`, en el frame `/map`): el heatmap de similitud CLIP de la categoría consultada, normalizado a `[0, 100]` y alineado con la celdilla del `occupancy_grid` del bridge.
- `/vlmap/candidates` (`geometry_msgs/PoseArray`, `latched`, en `/map`): los N máximos del heatmap (con supresión de no-máximos) como poses 2D para visualizar como flechas en RViz y, en su día, alimentar a `move_base`.

Decisión arquitectónica. Para preservar la separación `tfg-sim` (pesado: `torch` + `CLIP` + `VLMaps`) y `tfg-ros` (ligero: `ROS Noetic` + utilidades `NumPy`) se descarta cargar el `vlmaps.h5df` dentro del contenedor ROS. En su lugar se adopta un contrato *file-backed*: `tfg-sim` escribe heatmaps precomputados como `<categoría>.npz` (mismo grid que `occupancy.npz`) en un volumen compartido, y el nodo ROS los lee bajo demanda. Esto mantiene el contenedor ROS sin dependencias pesadas y es simétrico con cómo el `habitat_ros_bridge` ya consume el mapa y la pose.

Ficheros nuevos / modificados.

- *Nuevo*: `ros_ws/src/vlmap_semantic_server/src/vlmap_semantic_server/heatmap.py` con `load_heatmap_array`, `normalize_to_occupancy` y `extract_top_candidates`.
- *Modificado*: `vlmap_semantic_server_node` para instanciar publicadores `latched`, envolver `_handle_query_room` en `_handle_query_room_and_visualize` y aceptar los nuevos parámetros `heatmap_dir`, `map_frame` y `candidates_n`.
- *Modificado*: `package.xml` del semantic server añade dependencias `nav_msgs` y `geometry_msgs`.
- *Modificado*: `minimal.launch` expone `heatmap_dir` y `map_frame`.

Invocación de referencia.

```
roslaunch vlmap_bringup minimal.launch
  occupancy_path:=/shared/hssd/scene_X/occupancy.npz
  room_centroids_path:=/shared/hssd/scene_X/centroids.json
  heatmap_dir:=/shared/hssd/scene_X/heatmaps
  map_resolution:=0.05
```


2026-05-13 (cont.) — Cierre del bucle file-backed. Se completa el lado `tfg-sim` del contrato y los displays de RViz:

- *Nuevo paquete* `src/vlmap_ros_export/` con `heatmap_dumper.py` (NumPy puro: `pool_voxel_scores_to_numpy`, `dump_heatmap_numpy`, `normalize_heatmap`) y un CLI `vlmap_ros_export.cli` que envuelve `VlMap.load_map` y `get_lseg_score` para escribir un `<categoria>.npy` por categoría en el volumen compartido. Los módulos pesados (`torch`, `CLIP`, `h5py`) se importan de forma perezosa, así que el paquete sigue siendo importable desde `tfg-ros` si hiciera falta.
- *Nuevas tests* `tests/test_heatmap_dumper.py` (9 casos) y `tests/test_ros_heatmap_topics.py` (9 casos) cubren pooling, NMS, normalización, sanitización de nombre y rotación a la convención del bridge. **18/18 verde.**
- *Modificado:* `minimal.rviz` añade tres displays: Map sobre `/map`, Map (esquema `costmap`) sobre `/vlmap/heatmap` y PoseArray roja sobre `/vlmap/candidates`.

Pipeline completo end-to-end.

1. En `tfg-sim`: `python -m vlmap_ros_export.cli -data-dir <vlmap_dir> -output-dir /shared/<scene>/heatmaps -categories mug bottle laptop drill.`
2. En `tfg-ros`: `roslaunch vlmap_bringup minimal.launch occupancy_path:=... room_centroids_path:=... heatmap_dir:=/shared/<scene>/heatmaps.`
3. En RViz: `rosservice call /vlmap/query_room "category: 'mug'"` → aparecen el heatmap y las flechas de candidatos sobre el mapa de la escena.

2026-05-13 (cont.) — Captura de VLMaps desde ROS. Se añade el paquete `vlmap_capture` para construir un dataset VLMaps desde un mundo Gazebo (`small_house` o `wrs`) con la HSR teleoperada por teclado:

- *Nodo* `vlmap_capture_node`: sincroniza (`message_filters.ApproximateTimeSynchronizer`) `/hsrb/head_rgb_d_sensor/rgb/image_rect_color` + `depth_registered/image_rect_raw`, consulta TF `map→base_link` y vuelca a disco en el layout exacto que espera `VlMapBuilder`: `rgb/<id>.png`, `depth/<id>.npy` (float32 metros) y `poses.txt` (`tx ty tz qx qy qz qw` por línea). Adicionalmente escribe `meta.json` con los parámetros y el `camera_info`.
- *Submuestreo* por desplazamiento o rotación (`min_distance_m=0.20m`, `min_angle_rad≈20°`) para evitar redundancia espacial.
- *Servicios* `/vlmap_capture/start` y `/vlmap_capture/stop` (`std_srvs/Trigger`) para pausar y reanudar sin reiniciar el nodo.
- *Lógica testeable* extraída a `vlmap_capture/downsample.py` (`PoseSample`, `quaternion_yaw`, `should_accept_frame`, `format_pose_line`) — sin dependencia de `rospy`.
- *Tests offline* `tests/test_vlmap_capture_downsample.py` (9/9 verde) cubren yaw, primera aceptación, umbrales, wraparound y serialización.
- *Launch* `vlmap_bringup/launch/vlmap_capture_session.launch` que arranca mundo + HSR + teleop teclado + nodo de captura + RViz en una sola invocación.

Flujo end-to-end captura → heatmap.

1. En `tfg-ros`: `roslaunch vlmap_bringup vlmap_capture_session.launch world:=small_house run_id:=run01`. Conducir la HSR por todas las habitaciones; cuando termine, `rosservice call /vlmap_capture/stop "{}"`.
2. En `tfg-sim`: usar el `VlMapBuilder` con `pose_type=mobile_base` apuntando a `/shared/captures/small_h`

para producir `vmap/vlmaps.h5df`.

3. En `tfg-sim`: `python -m vmap_ros_export.cli -data-dir /shared/captures/small_house/run01 -output-dir /shared/heatmaps/small_house -categories mug bottle ....`
4. En `tfg-ros`: relanzar `minimal.launch` apuntando `occupancy_path` y `heatmap_dir` al volcado, y consultar `/vmap/query_room` desde RViz.

2026-06-07: validación semántica del VLMap capturado. El flujo captura ROS/Gazebo → construcción LSeg+CLIP → carga en `tfg-sim` queda validado con el mismo criterio nativo de VLMaps. La medición ya no usa heatmaps crudos de una sola categoría, sino el índice multiclase: `init_categories(mp3dcat)` seguido de `index_map`, donde cada vóxel gana una única etiqueta frente al resto del vocabulario y el fondo. Esto mantiene la inteligencia semántica en `tfg-sim`; ROS sigue siendo únicamente backend de navegación y sensores.

Sobre la captura `small_house/run_20260607_174653` se obtienen 25 629 celdas ocupadas, IoU medio 0.00010 e IoU máximo 0.00126 entre categorías de habitaciones distintas. Como referencia, el mapa HSSD 102344280_0 obtiene IoU medio 0.0120 e IoU máximo 0.1655, este último entre `table` y `chair`. La captura ROS/Gazebo separa cama, fregadero, zona de sofá y sillas con máscaras prácticamente disjuntas; la clase `toilet` aparece vacía porque no está presente de forma estable en esa captura.

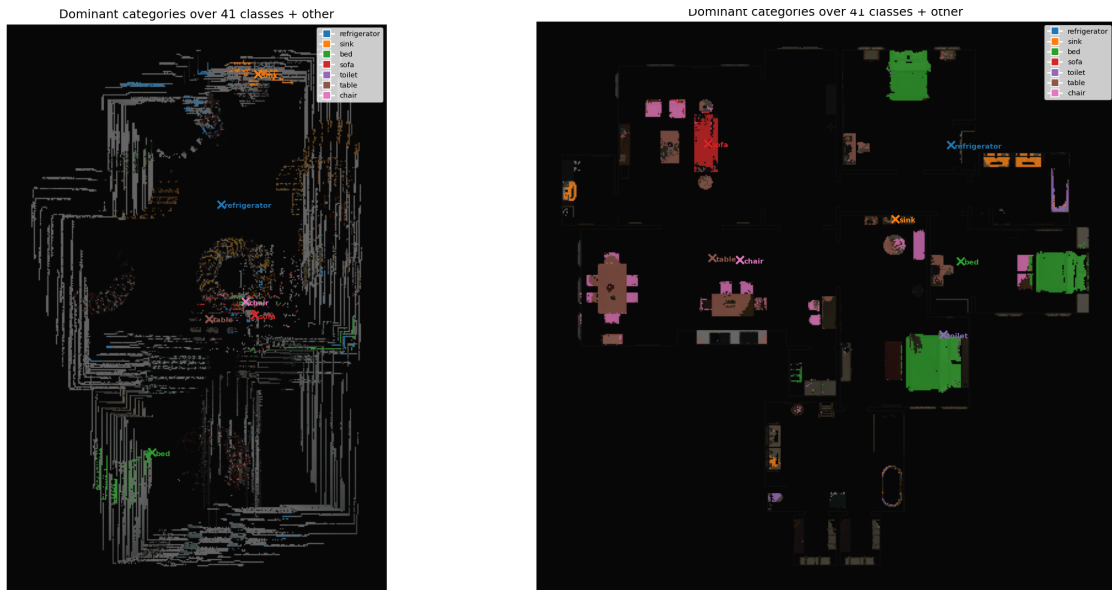


Figura 1: Validación sin interfaz gráfica del índice multiclase para el mapa capturado desde ROS/Gazebo (izquierda) y para un mapa HSSD de referencia (derecha).

Pendiente para la siguiente iteración.

- Validar la captura real contra `small_house` y `wrs` (verificación visual de que las nubes de puntos cubren las habitaciones esperadas).
- Ajustar `base2cam_tf` y `base_transform` en la configuración de VLMaps para la convención de la HSR (la actual está calibrada para Habitat).
- Conectar los candidatos publicados a `move_base` para cerrar el bucle de navegación dentro de Gazebo.